T6 · Cloud Computing e DevOps Avançado

Estratégia de CD: deploy e rollback

Biblioteca (Spring Boot 4.0.6 · Java 21) em Kubernetes

Keltton, Felipe, Daniel · Prof. Ranghetti

Contexto e restrições

Artefato

Imagem no GHCR com a tag do commit: promove e volta para uma versão exata.

Execução

Kubernetes: Deployment + Service bastam, sem ferramentas extras.

Banco H2 em memória

Só 1 réplica: cada pod teria dados próprios.

Sessão em memória

Na troca de versão, o usuário entra de novo.

Blue/Green, Canary ou Rolling

Critério	Blue/Green (escolhida)	Canary	Rolling update
1 réplica com H2	✅ Uma versão por vez	❌ Exige 2+ réplicas, bancos diferentes	⚠️ Funciona
Testar antes do tráfego	✅ Pelo serviço preview	✅ Com parte do tráfego real	❌ Não
Rollback	✅ Segundos	✅ Segundos	⚠️ Minutos
Ferramentas extras	✅ Só kubectl	❌ Service mesh ou Ingress com peso	✅ Nenhuma
Custo	⚠️ 2 pods por ~15 min	⚠️ Réplicas extras	✅ Menor

Justificativa de negócio: rollback em segundos limita o impacto de uma versão com defeito; o custo extra é um pod por cerca de 15 min.

Fluxo de deploy

1 Imagem publicada

biblioteca:<sha> no registro, após o CI do T5.

2 Deploy na cor inativa

1 réplica; o tráfego continua na versão atual.

3 Pod pronto?

Readiness OK em 180 s; senão, aborta.

4 Smoke tests

Via biblioteca-preview; falhou, aborta.

5 Troca do seletor

O Service aponta para a nova cor.

6 Janela de 15 min

Tudo OK: finalizar.sh. Critério violado: rollback.sh.

Critérios de sucesso

Antes da troca (automático)

1. Pod pronto em até 180 s
2. Readiness (porta 8081) = 200
3. GET /login = 200
4. POST /api/auth/login devolve JWT
5. GET /api/livros: 200 ou 404, nunca 401/403

Qualquer falha aborta; o tráfego fica na versão atual.

Depois da troca (15 min)

Indicador	Sucesso	Rollback se
Disponibilidade	≥ 99,5%	< 99,5% ou queima rápida
Latência p95	< 500 ms	≥ 500 ms por 5 min
Pod	up = 1, 0 reinícios	Alerta de indisponível ou reiniciando
Funcional	Login e cadastro OK	Falha em fluxo principal

Rollback: primeiro volta, depois investiga

Situação	Ação	RTO
Cor anterior ainda ligada	rollback.sh troca o seletor	< 30 s
Depois do finalizar.sh	rollback.sh religa a cor anterior e troca	≈ 2 min
Imagem anterior fora do ar	deploy-bluegreen.sh <sha-anterior>	≈ 3 min
Primeiro deploy	Script recusa e avisa; corrigir e refazer	—

Depois: confirmar no painel, manter a cor com defeito ligada para ler logs, registrar o incidente, corrigir e passar de novo pelo CI.

Limitações conhecidas e evolução

Limitação	Consequência	Mitigação futura
H2 em memória	Dados se perdem em toda troca, inclusive no rollback	Banco gerenciado (RDS, Cloud SQL, Azure SQL)
Sessão em memória	Login web refeito após a troca	Spring Session + Redis, ou só JWT
Mudança de esquema	As duas cores precisam do mesmo esquema	Migrações expand/contract

Próximo passo: com banco externo e várias réplicas, Canary passa a ser viável.